

Nirmaanify AI

Complete Master Implementation Plan

Phase-by-Phase and Week-by-Week Development Roadmap

Product: Nirmaanify AI

Tagline: Imagine. Build. Launch.

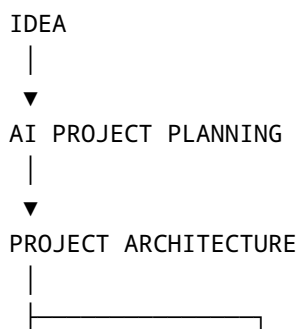
1. PRODUCT VISION

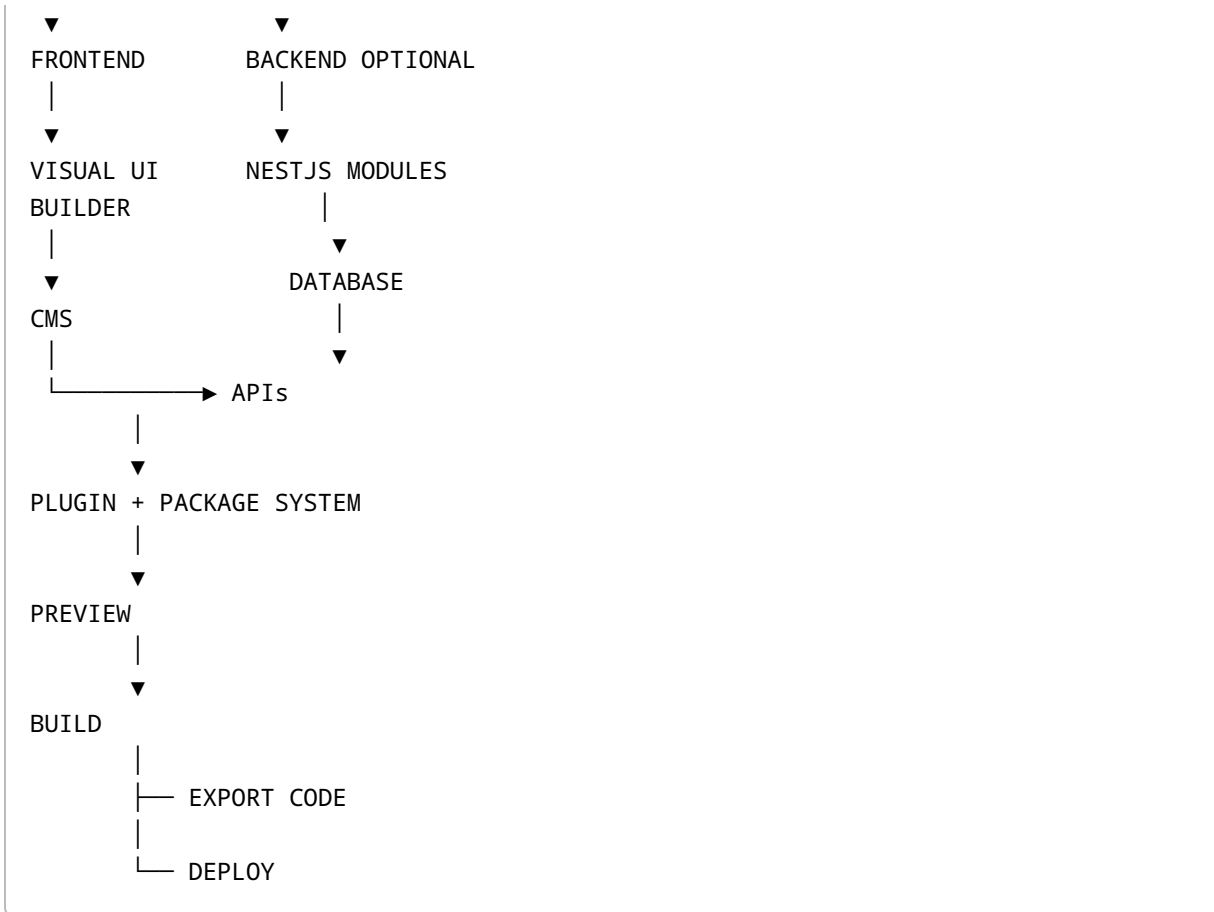
Nirmaanify AI is an AI-powered platform that enables users to build, manage, customize, export, and deploy websites and full-stack applications.

The platform combines:

- AI application generation
- Visual drag-and-drop website building
- Component-based development
- CMS generation and management
- Optional backend generation using NestJS
- Database and API management
- Plugin architecture
- NPM package/library integration
- User-selectable UI libraries
- User-selectable animation libraries
- Project preview
- Code generation
- Project export
- Deployment integration

The central product flow is:





2. THE MOST IMPORTANT ARCHITECTURAL RULE

Nirmaanify AI contains two different design environments.

Environment A — Nirmaanify Platform

This includes:

- Dashboard
- Studio
- CMS
- Backend
- Database
- API
- Plugins
- Packages
- AI Assistant
- Projects

Deployments
Settings

These must always follow the protected:

Nirmaanify Design System

Every future feature and AI-generated internal UI must use:

- Nirmaanify colors
- Nirmaanify typography
- Nirmaanify spacing
- Nirmaanify components
- Nirmaanify layout patterns
- Nirmaanify interaction patterns
- Nirmaanify accessibility rules

Environment B — User Project

The user-generated website or application can have its own:

Logo
Brand
Colors
Typography
Theme
Components
Layout
Animations
UI Library

Therefore:

NIRMAANIFY PLATFORM
=
STRICT DESIGN CONSISTENCY

USER PROJECT
=
CREATIVE FREEDOM

3. COMPLETE TECHNOLOGY ARCHITECTURE

Monorepo

Use:

```
Turborepo
+
pnpm workspaces
```

Recommended architecture:

```
nirmaanify-ai/
|
├── apps/
|   |
|   ├── web/
|   |   └── Main Nirmaanify Platform
|   |
|   ├── api/
|   |   └── NestJS API
|   |
|   ├── worker/
|   |   └── Background Jobs
|   |
|   ├── preview/
|   |   └── Project Preview Runtime
|   |
|   └── docs/
|       └── Documentation
|
├── packages/
|   |
|   ├── design-tokens/
|   ├── ui/
|   ├── icons/
|   ├── types/
|   ├── config/
|   ├── ai/
|   ├── builder-core/
|   ├── component-registry/
|   ├── project-schema/
|   └── code-generator/
```

```
|   |   | plugin-sdk/
|   |   | project-runtime/
|   |   | backend-generator/
|   |
|   |--- infrastructure/
|   |   |   | docker/
|   |   |   | nginx/
|   |   |   | terraform/
|   |
|   |--- docs/
|   |   | architecture/
|   |   | design-system/
|   |   | ai-agents/
|   |   | plugin-sdk/
```

4. RECOMMENDED CORE STACK

Frontend

```
Next.js
TypeScript
React
Tailwind CSS
shadcn/ui
Radix UI
TanStack Query
Zustand
React Hook Form
Zod
Lucide React
```

Drag and Drop

```
dnd-kit
```

Visual Builder

```
React
JSON Schema
Component Registry
```

Renderer Engine
Property Inspector
Undo/Redo History

Backend

NestJS
TypeScript
Prisma
PostgreSQL
Redis
BullMQ

Authentication

Better Auth / Auth.js-compatible architecture
JWT / Secure Session
OAuth

AI

LLM Provider Abstraction
Agent Orchestrator
Tool Calling
Structured JSON Output
RAG for documentation and project context

Storage

PostgreSQL
Redis
S3 Compatible Object Storage

Infrastructure

Docker
Docker Compose
GitHub Actions

5. DEVELOPMENT ROADMAP OVERVIEW

PHASE 1 Weeks 1-4

Design System Foundation

PHASE 2 Weeks 5-6

Monorepo and Technical Foundation

PHASE 3 Weeks 7-9

Authentication and Workspaces

PHASE 4 Weeks 10-12

Project Management and AI Project Planning

PHASE 5 Weeks 13-16

Project Schema and Frontend Architecture Engine

PHASE 6 Weeks 17-20

Visual Drag-and-Drop Studio

PHASE 7 Weeks 21-23

CMS and Content Management

PHASE 8 Weeks 24-27

Backend Builder and NestJS Generation

PHASE 9 Weeks 28-29

Database and API Builder

PHASE 10 Weeks 30-31

Package, Library and Plugin System

PHASE 11 Weeks 32-33

AI Agent Orchestration

PHASE 12 Weeks 34-35

Preview, Build, Export and Deployment

PHASE 13 Week 36

Testing, Security and MVP Launch

PHASE 1 — DESIGN SYSTEM FOUNDATION

Duration

Week 1–4

Goal

Create the permanent Nirmaanify AI brand and UI foundation before major application features are built.

No feature should create random styling.

WEEK 1 — BRAND IDENTITY

Tasks

- [] Finalize Nirmaanify AI logo
- [] Create Modular N icon
- [] Create horizontal logo
- [] Create wordmark
- [] Create favicon
- [] Create application icon
- [] Create light logo
- [] Create dark logo
- [] Create SVG assets

Brand Definition

Name:

Nirmaanify AI

Meaning:

Nirmaan + Ify

Tagline:

Imagine. Build. Launch.

Personality:

Intelligent

Professional

Creative

Modern
Precise
Powerful

Deliverable

Nirmaanify Brand Kit v1

WEEK 2 — DESIGN TOKENS

Create:

packages/design-tokens/

Implement:

- [] Color tokens
- [] Typography tokens
- [] Spacing tokens
- [] Border radius
- [] Shadow tokens
- [] Motion tokens
- [] Breakpoints
- [] Z-index tokens
- [] Light theme
- [] Dark theme

Color Direction

Nirmaan Indigo
#635BFF

Build Blue
#3B82F6

AI Violet
#8B5CF6

Launch Cyan
#22D3EE

Typography

Display:
Geist

Application UI:
Inter

Code:
Geist Mono

Deliverable

Design Tokens v1

WEEK 3 — CORE COMPONENT SYSTEM

Create:

packages/ui/

Build:

```
[ ] Button
[ ] Icon Button
[ ] Input
[ ] Textarea
[ ] Select
[ ] Checkbox
[ ] Switch
[ ] Badge
[ ] Card
[ ] Avatar
[ ] Separator
[ ] Skeleton
```

Every component must support:

Light Mode
Dark Mode

Disabled State
Loading State
Focus State
Keyboard Accessibility

WEEK 4 — PATTERNS AND DESIGN GOVERNANCE

Build:

- [] Application Shell
- [] Sidebar
- [] Topbar
- [] Page Header
- [] Empty State
- [] Error State
- [] Loading State
- [] Success Feedback
- [] Toast System
- [] Dialog
- [] Drawer
- [] Dropdown
- [] Tabs

Create:

DESIGN_SYSTEM.md
COMPONENT_RULES.md
UI_PATTERNS.md
AI_UI_RULES.md
FEATURE_IMPLEMENTATION_RULES.md

Critical AI Rule

Every AI agent must first determine:

designContext

Possible values:

```
platform
project
```

Deliverable

```
Nirmaanify Design System v1
```

PHASE 2 — MONOREPO AND TECHNICAL FOUNDATION

Duration

Week 5-6

WEEK 5 — MONOREPO SETUP

Create:

```
[ ] Turborepo
[ ] pnpm workspace
[ ] Next.js application
[ ] NestJS application
[ ] Shared TypeScript config
[ ] ESLint
[ ] Prettier
[ ] Environment configuration
[ ] Docker configuration
```

Configure:

```
apps/web
apps/api
apps/worker
packages/ui
packages/types
packages/design-tokens
packages/config
```

WEEK 6 — DATABASE AND INFRASTRUCTURE FOUNDATION

Set up:

```
PostgreSQL
Prisma
Redis
BullMQ
S3 Storage abstraction
Docker Compose
```

Create initial infrastructure models:

```
User
Workspace
Project
ProjectMember
Organization
AuditLog
```

Implement:

```
[ ] Database migrations
[ ] Health checks
[ ] Logging
[ ] Error handling
[ ] API response standard
[ ] Environment validation
```

Deliverable

```
Working Monorepo Foundation
```

PHASE 3 — AUTHENTICATION AND WORKSPACES

Duration

Week 7-9

WEEK 7 — AUTHENTICATION

Implement:

```
[ ] Register
[ ] Login
[ ] Logout
[ ] Email verification
[ ] Password reset
[ ] Session management
[ ] Protected routes
```

Architecture:

```
User
└─ Workspaces
   └─ Projects
```

WEEK 8 — WORKSPACES AND ORGANIZATIONS

Implement:

```
[ ] Personal workspace
[ ] Organization workspace
[ ] Invite members
[ ] Remove members
[ ] Roles
[ ] Permissions
```

Initial roles:

```
Owner
Admin
Developer
Editor
Viewer
```

WEEK 9 — APPLICATION DASHBOARD

Build using the Phase 1 Design System:

- ☐ Dashboard
- ☐ Workspace switcher
- ☐ Recent projects
- ☐ Project cards
- ☐ Quick actions
- ☐ AI project prompt
- ☐ Notifications

Deliverable

Authenticated Nirmaanify Dashboard

PHASE 4 — PROJECT MANAGEMENT AND AI PROJECT PLANNING

Duration

Week 10–12

WEEK 10 — PROJECT MANAGEMENT

Implement:

- ☐ Create project
- ☐ Edit project
- ☐ Delete project
- ☐ Duplicate project
- ☐ Archive project
- ☐ Project settings

Project types:

Website
Blog
E-commerce
Portfolio
Dashboard
SaaS
Custom Application

WEEK 11 — AI PROJECT PLANNER

User prompt:

"I want to create an online clothing store."

AI should generate:

Project Name
Project Type
Pages
Features
Components
Required Packages
Backend Requirements
Database Requirements
CMS Requirements
Plugin Recommendations
Architecture Plan

Example:

AI PROJECT PLAN

Project:
Fashion Store

Frontend:
Next.js

Pages:
Home
Products

Product Details
Cart
Checkout

Backend:
Enabled

Modules:
Products
Orders
Users
Payments

Database:
PostgreSQL

WEEK 12 — PROJECT PLAN APPROVAL FLOW

Implement:

```
graph TD;
    A[AI Plan] --> B[User Review];
    B --> C[Modify Plan];
    C --> D[Approve];
    D --> E[Generate Project];
```

The AI must not automatically generate large projects without confirmation.

Deliverable

AI-Powered Project Creation Flow

PHASE 5 — PROJECT SCHEMA AND FRONTEND ARCHITECTURE ENGINE

Duration

Week 13–16

This is one of the most important technical phases.

The visual editor should not directly save React code.

Instead:

```
VISUAL EDITOR
  ↓
PROJECT JSON SCHEMA
  ↓
COMPONENT TREE
  ↓
RENDERER
  ↓
PREVIEW
  ↓
CODE GENERATOR
```

WEEK 13 — PROJECT SCHEMA

Design:

```
Project
├── Settings
├── Theme
├── Pages
│   └── Component Tree
├── Assets
├── Data Sources
├── Packages
├── Plugins
└── Backend Configuration
```

Example:

```
{
  "type": "hero",
  "props": {
    "title": "Build Faster",
    "subtitle": "Create with AI"
  },
  "children": []
}
```

WEEK 14 — COMPONENT REGISTRY

Create:

```
packages/component-registry/
```

Each component contains:

```
Component ID
Name
Category
React Component
Props Schema
Default Props
Allowed Children
Inspector Controls
Required Packages
```

Example:

```
hero
navbar
button
text
image
grid
form
product-card
```

WEEK 15 — RENDERING ENGINE

Build:

```
Component Tree
  ↓
Component Resolver
  ↓
React Renderer
```

Implement:

```
[ ] Dynamic component rendering
[ ] Props rendering
[ ] Nested components
[ ] Error boundary
[ ] Missing component fallback
```

WEEK 16 — HISTORY AND VALIDATION

Implement:

```
[ ] Undo
[ ] Redo
[ ] Schema validation
[ ] Project validation
[ ] Auto-save
[ ] Draft versions
```

Deliverable

```
Project Architecture Engine v1
```

PHASE 6 — VISUAL DRAG-AND-DROP STUDIO

Duration

Week 17-20

WEEK 17 — STUDIO APP SHELL

Build:

```
Top Toolbar
Component Sidebar
Canvas
Inspector Panel
AI Command Bar
```

Layout:

```
COMPONENTS | CANVAS | INSPECTOR
```

WEEK 18 — DRAG AND DROP

Implement:

```
[ ] Drag component into canvas
[ ] Drop zones
[ ] Reorder components
[ ] Nested components
[ ] Delete
[ ] Duplicate
[ ] Move
```

Use:

```
dnd-kit
```

WEEK 19 — PROPERTY INSPECTOR

Implement:

```
Content
Style
Layout
Responsive
Interactions
```

Examples:

```
Text:
Content
Font size
Weight

Button:
Text
Link
Variant
Size

Container:
Padding
Gap
Alignment
Width
```

WEEK 20 — RESPONSIVE PREVIEW

Add:

```
Desktop
Tablet
Mobile
```

Implement:

```
[ ] Preview switching
[ ] Responsive values
```

- [] Canvas scaling
- [] Selection overlay
- [] Component toolbar

Deliverable

Functional Drag-and-Drop Website Builder

PHASE 7 — CMS AND CONTENT MANAGEMENT

Duration

Week 21-23

WEEK 21 — CMS COLLECTIONS

Allow:

- Posts
- Products
- Categories
- Authors
- Custom Collections

Create collection builder:

- Field Name
- Field Type
- Required
- Validation
- Default Value

Field types:

- Text
- Rich Text
- Number
- Boolean

Date
Image
File
Select
Relation
JSON

WEEK 22 — CONTENT MANAGEMENT

Build:

[] Content list
[] Create content
[] Edit content
[] Delete content
[] Draft
[] Publish
[] Scheduled publishing

WEEK 23 — FRONTEND CMS BINDING

Connect:

CMS Data
↓
Data Source
↓
Visual Component
↓
Website Rendering

Example:

Blog Collection
↓
Blog List Component
↓
Generated Blog Page

Deliverable

Dynamic CMS System

PHASE 8 — BACKEND BUILDER AND NESTJS GENERATION

Duration

Week 24–27

Backend should be optional.

User can enable:

Enable Backend

If disabled:

Frontend-only project

If enabled:

Frontend
+
NestJS Backend
+
Database
+
API

WEEK 24 — BACKEND PROJECT CONFIGURATION

Build backend dashboard:

Overview
Modules
Database
API
Environment

Implement:

☐ Enable backend
☐ Disable backend
☐ Backend settings
☐ Module list

WEEK 25 — BACKEND MODULE BUILDER

Predefined modules:

Authentication
Users
Products
Categories
Orders
Payments
Blog
Notifications
Uploads

User can:

Enable
Disable
Configure
Generate

WEEK 26 — NESTJS CODE GENERATION

Architecture:

BACKEND SCHEMA
↓
MODULE GENERATOR
↓
NESTJS FILES

Generate:

Module
Controller
Service
DTO
Validation
Prisma integration
Guards
Permissions

WEEK 27 — BACKEND CONNECTION

Connect:

Frontend Components
↓
API Data Sources
↓
NestJS API
↓
PostgreSQL

Deliverable

Optional Full-Stack Project Generation

PHASE 9 — DATABASE AND API BUILDER

Duration

Week 28-29

WEEK 28 — DATABASE MODEL BUILDER

Create:

Data Models

Example:

```
Product  
  
name  
price  
description  
categoryId  
stock  
createdAt
```

Support:

```
String  
Number  
Boolean  
Date  
Enum  
JSON  
Relation
```

Start with table configuration first.

Later add:

Visual ER Diagram

WEEK 29 — API BUILDER

Generate:

GET
POST
PATCH
DELETE

Allow:

Authentication
Permissions
Validation
Pagination
Filtering
Sorting

Deliverable

Visual Data and API Builder

PHASE 10 — PACKAGE, LIBRARY AND PLUGIN SYSTEM

Duration

Week 30-31

WEEK 30 — PACKAGE AND LIBRARY MANAGEMENT

Users can choose libraries.

Examples:

UI

shadcn/ui
Material UI
Custom Components

Animation

Framer Motion
GSAP
Motion One

Forms

React Hook Form
Formik

Architecture:

```
PROJECT
  ↓
PACKAGE MANIFEST
  ↓
DEPENDENCY MANAGER
  ↓
PROJECT PACKAGE.JSON
```

Implement compatibility checks.

```
Package Selected
  ↓
Version Check
  ↓
Framework Check
  ↓
Conflict Check
  ↓
Install
```

WEEK 31 — PLUGIN SDK AND MARKETPLACE

Create:

Plugin Manifest
Plugin Permissions
Plugin API

Plugin Lifecycle
Plugin Sandbox Strategy

Categories:

UI
Animation
Payments
Authentication
Analytics
SEO
Forms
CMS
Deployment

Plugin structure:

plugin.json
frontend/
backend/
generator/
configuration/

Deliverable

Plugin and Package Ecosystem Foundation

PHASE 11 — AI AGENT ORCHESTRATION

Duration

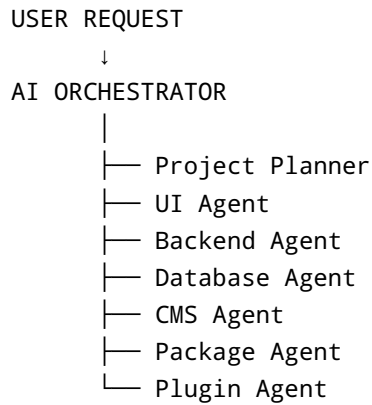
Week 32-33

Do not create one giant AI agent.

Use specialized agents.

WEEK 32 — AI ORCHESTRATOR

Architecture:



The orchestrator controls:

Task routing
Context
Dependencies
Progress
Approval
Errors
Retries

WEEK 33 — AI DESIGN CONTEXT AND MEMORY

Every UI task receives:

designContext

platform

means:

Use Nirmaanify Design System

project

means:

Use User Project Theme

Add:

Project Context
Architecture Context
Component Context
Package Context
Backend Context
Conversation Context

Deliverable

Multi-Agent AI Development System

PHASE 12 — PREVIEW, BUILD, EXPORT AND DEPLOYMENT

Duration

Week 34–35

WEEK 34 — PREVIEW AND BUILD

Build:

Live Preview
Build Validation
Error Logs
Build Logs
Environment Validation

Flow:

```
PROJECT SCHEMA
  ↓
GENERATED PROJECT
  ↓
BUILD
  ↓
PREVIEW
```

WEEK 35 — CODE EXPORT AND DEPLOYMENT

Allow export:

```
Download ZIP
Git Repository
GitHub Push
```

Generated structure:

```
my-project/
|
├── frontend/
│   └── Next.js
|
├── backend/
│   └── NestJS
|
├── docker-compose.yml
|
├── README.md
|
└── .env.example
```

Deployment options can later include:

```
Vercel
Render
Railway
```

Docker
Self-hosted

Deliverable

Exportable and Deployable Projects

PHASE 13 — TESTING, SECURITY AND MVP LAUNCH

Duration

Week 36

Testing

Implement:

Unit Tests
Integration Tests
E2E Tests
Schema Tests
Component Tests
API Tests
Visual Regression Tests

Critical flows:

User Registration
Project Creation
AI Planning
Drag and Drop
CMS
Backend Generation
API Generation
Export

Security

Review:

- Authentication
- Authorization
- Workspace Isolation
- Project Access
- API Permissions
- Plugin Permissions
- Package Security
- Secret Storage
- Rate Limiting
- Input Validation
- Generated Code Isolation

MVP RELEASE CHECKLIST

- ☐ Authentication
- ☐ Workspace
- ☐ Project Creation
- ☐ AI Project Planner
- ☐ Component Registry
- ☐ Visual Builder
- ☐ CMS
- ☐ Optional Backend
- ☐ Database Models
- ☐ API Generation
- ☐ Package Selection
- ☐ Plugin Foundation
- ☐ AI Agents
- ☐ Preview
- ☐ Export

Final Deliverable

Nirmaanify AI MVP v1.0

6. MVP PRIORITY LEVELS

To avoid making the project too large, features should be prioritized.

P0 — MUST HAVE

- Design System
- Authentication
- Projects
- AI Planning
- Visual Builder
- Component Registry
- Project Schema
- Preview
- Export

P1 — HIGH PRIORITY

- CMS
- Backend Toggle
- NestJS Basic Generation
- Database Models
- API Generation
- Package Selection

P2 — AFTER MVP

- Advanced Plugin Marketplace
- Advanced AI Agents
- GitHub Integration
- Deployment Providers
- Collaboration
- Real-time Multiplayer
- ER Diagram
- Advanced Animation Editor

7. RECOMMENDED DAILY WORKFLOW

Every development task should follow:

1. SELECT FEATURE
- ↓
2. WRITE REQUIREMENTS
- ↓
3. CHECK ARCHITECTURE
- ↓
4. CHECK DESIGN SYSTEM
- ↓
5. CHECK EXISTING COMPONENTS
- ↓
6. IMPLEMENT BACKEND
- ↓
7. IMPLEMENT FRONTEND
- ↓
8. CONNECT API
- ↓
9. TEST
- ↓
10. DOCUMENT
- ↓
11. MERGE

For UI work:

- FEATURE REQUEST
- ↓
- CHECK DESIGN CONTEXT
- ↓
- PLATFORM OR PROJECT?
- ↓
- CHECK EXISTING PATTERNS
- ↓
- REUSE COMPONENTS
- ↓
- CREATE ONLY MISSING COMPONENTS
- ↓
- TEST LIGHT/DARK MODE
- ↓
- RESPONSIVE TEST

↓
ACCESSIBILITY TEST

8. WEEKLY DEVELOPMENT RHYTHM

A recommended workflow for each week:

Monday

Planning
Architecture
Task Breakdown

Tuesday–Wednesday

Main Implementation

Thursday

Integration
Bug Fixing
Edge Cases

Friday

Testing
Refactoring
Documentation
Demo

Weekend or Final Day

Review
Backlog Update
Next Week Planning

9. DEVELOPMENT RULES

Rule 1

Do not build features directly into the main application without reusable architecture.

Build System
Not One-Time Code

Rule 2

Do not let AI generate arbitrary UI for the Nirmaanify platform.

All internal AI-generated UI must follow:

Nirmaanify Design System

Rule 3

Keep the project schema separate from generated code.

PROJECT STATE
≠
GENERATED REACT CODE

Use:

Project Schema
→ Renderer
→ Code Generator

Rule 4

Backend is optional.

Simple Website
→ Frontend Only

E-commerce
→ Frontend + Backend + Database

Blog
→ Frontend + CMS

SaaS
→ Full Stack

Rule 5

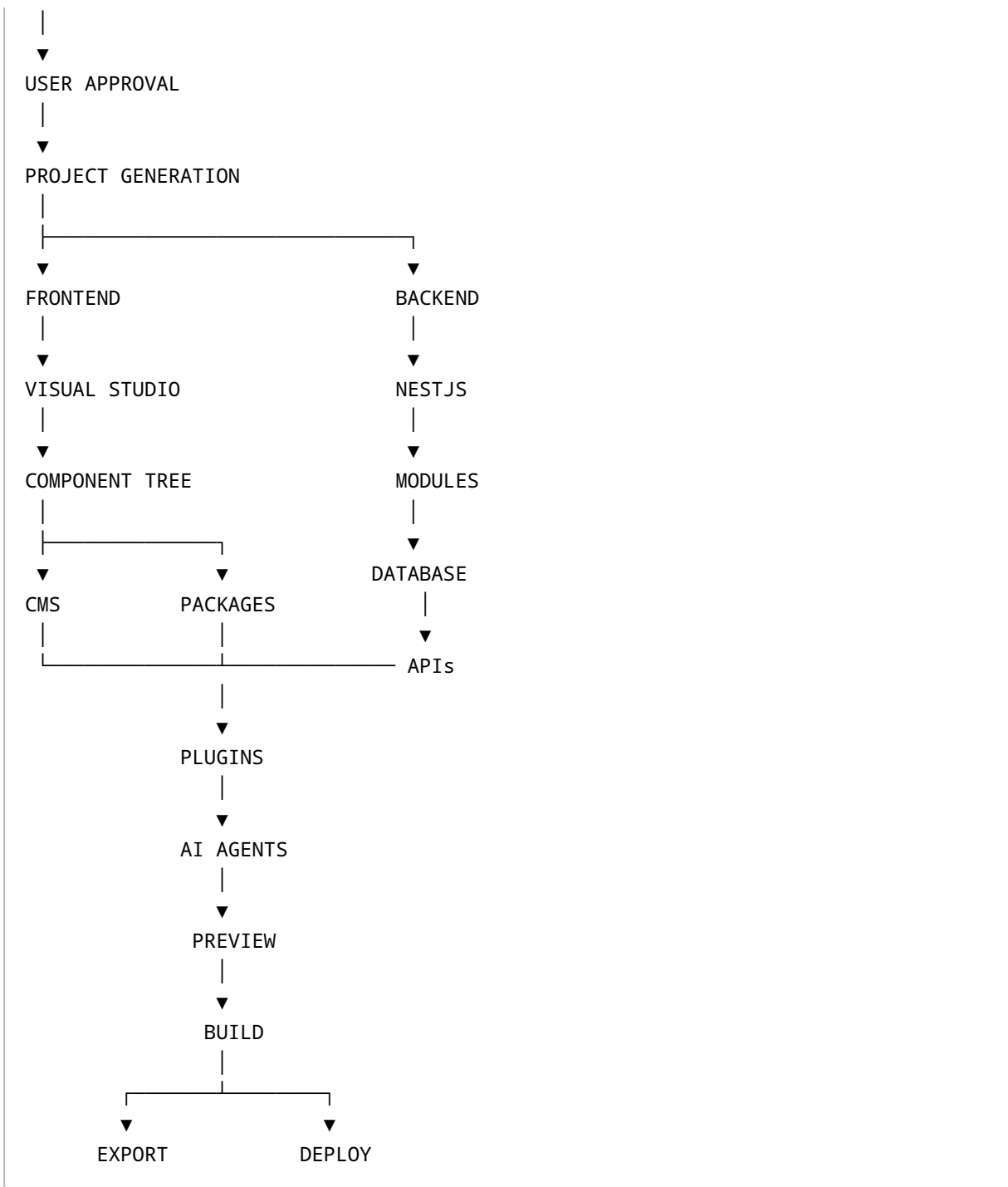
Every generated project owns its own design context.

Nirmaanify Platform
→ Fixed Design System

User Project
→ Custom Design System

10. FINAL MASTER PRODUCT FLOW

```
graph TD
  USER --> NIRMAANIFY_AI[NIRMAANIFY AI]
  NIRMAANIFY_AI --> CREATE_PROJECT[CREATE PROJECT]
  CREATE_PROJECT --> DESCRIBE_IDEA[DESCRIBE IDEA]
  DESCRIBE_IDEA --> AI_PROJECT_PLANNER[AI PROJECT PLANNER]
  AI_PROJECT_PLANNER --> ARCHITECTURE_PLAN[ARCHITECTURE PLAN]
```



FINAL 36-WEEK OUTCOME

At the end of this roadmap, Nirmaanify AI should have a working foundation for becoming an AI-powered full-stack application builder where users can:

- ✓ Describe an application using AI
- ✓ Generate a project plan
- ✓ Choose frontend libraries
- ✓ Choose animation packages
- ✓ Build pages visually
- ✓ Drag and drop components
- ✓ Edit component properties
- ✓ Create CMS collections
- ✓ Manage content
- ✓ Enable a NestJS backend
- ✓ Configure backend modules
- ✓ Create database models
- ✓ Generate APIs
- ✓ Install supported packages
- ✓ Install plugins
- ✓ Use specialized AI agents
- ✓ Preview applications
- ✓ Build projects
- ✓ Export source code
- ✓ Deploy applications

Most importantly, the platform itself remains visually consistent because every new internal feature follows:

NIRMAANIFY DESIGN TOKENS
↓
NIRMAANIFY COMPONENT SYSTEM

↓
NIRMAANIFY UI PATTERNS
↓
NIRMAANIFY DESIGN GOVERNANCE
↓
AI AGENT DESIGN CONTEXT
↓
CONSISTENT PLATFORM EXPERIENCE

While every website or application built by a customer remains:

Independent
Customizable
Brandable
Exportable

Final Product Principle

Nirmaanify AI builds the platform with consistency and builds user projects with freedom.